CodeLens

Code Intelligence Infrastructure for AI Agents

Final Project Report

Multitier Business Application Development

Submission Date: June 03, 2026

GitHub: github.com/emirrozerr/code-lens

Team Members

Emir Ozer — 72258

Mustafa Kaan Yavuz — 72639

Kadir Kutay Yümsek — 73283

1. Executive Summary

CodeLens is a three-tier business application that solves a real and growing problem in AI-assisted software development: AI coding agents lack structural understanding of codebases. When a developer asks Claude or Copilot about their codebase, the agent relies on text search — it finds keywords, not relationships. It cannot answer 'What would break if I change this method?' or 'What business domain does this class belong to?'

CodeLens fills this gap by parsing Java and Python repositories into a Neo4j knowledge graph — capturing classes, methods, call chains, inheritance, and conditional branches as first-class graph nodes and edges. It then applies community detection (Louvain algorithm) and a Groq LLM to automatically identify and name business domains within the codebase. The resulting graph is exposed through an MCP (Model Context Protocol) server, making all of this structural intelligence directly available to any MCP-compatible AI agent.

The system also ships a Next.js frontend with a streaming chat interface, force-directed graph visualization, and a full admin panel — allowing teams without a CLI to interact with the same underlying graph through a browser.

Key capabilities delivered:

- Multi-language AST parsing (Java, Python) via Tree-sitter
- Neo4j graph database with fulltext search and structural traversals
- Automated business domain discovery via Louvain community detection + Groq LLM
- MCP server with 7 tools, compatible with Claude Desktop, Claude Code, and Cursor
- FastAPI REST backend with JWT authentication and role-based access control
- Next.js 15 frontend: streaming chat, graph visualization, admin panel
- Incremental re-indexing via file system watcher

2. System Architecture

2.1 Three-Tier Overview

CodeLens strictly follows the three-tier architecture pattern required by the course. Each tier has a single, well-defined responsibility and communicates with adjacent tiers only through documented interfaces.

Tier	Implementation	Responsibility
Presentation	Next.js 15 (React 19, TypeScript)	User interfaces: chat, graph visualization, admin panel
Application	FastAPI + MCP Server (Python)	Business logic, auth, LLM orchestration, graph queries
Data	Neo4j Graph Database	Code graph persistence, admin data (users, repos, jobs)

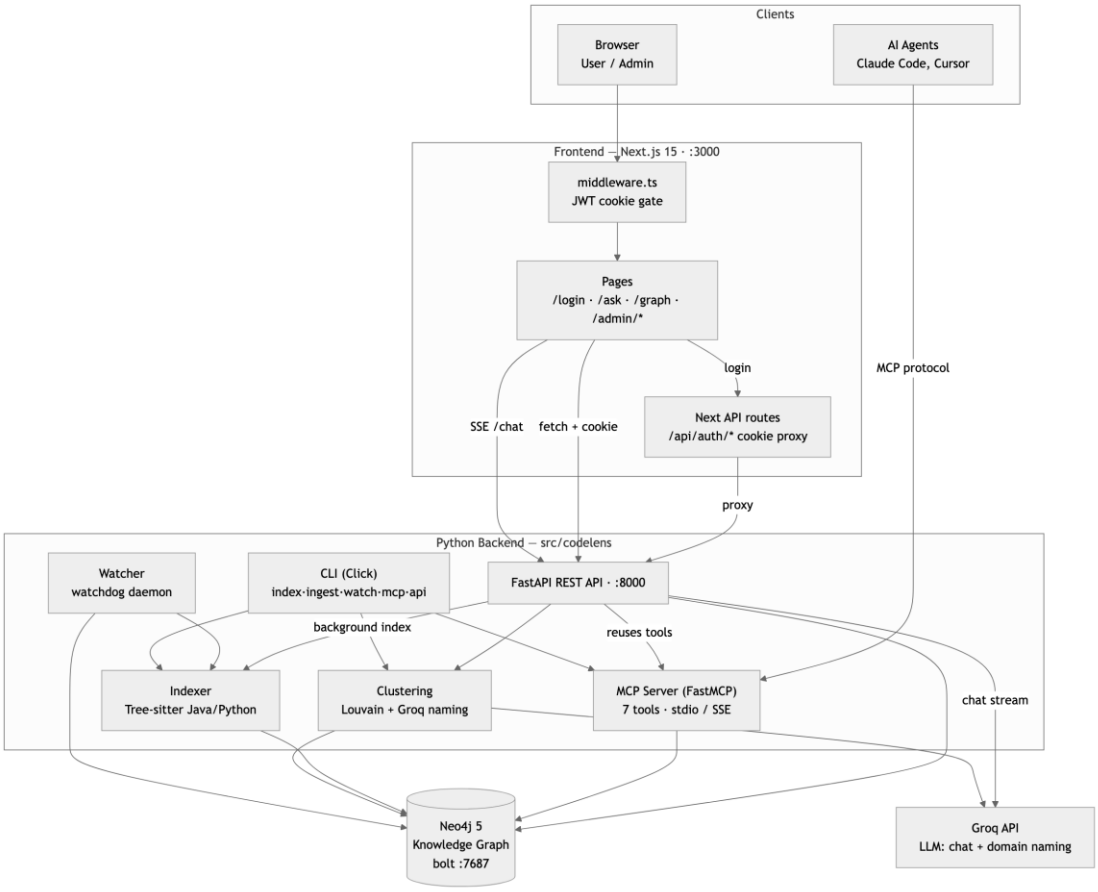

Figure 1: System Architecture — three-tier overview with all components and communication paths

2.2 Request Flow

The system supports two distinct client paths that share the same data tier:

- **AI Agent path:**

Claude Desktop or Claude Code launches the MCP server as a subprocess (stdio transport). The agent calls MCP tools (e.g., search_nodes, get_callers), which execute Cypher queries against Neo4j and return structured text results.

- **Browser path:**

The Next.js frontend makes HTTP requests to the FastAPI backend. The chat endpoint streams SSE tokens from Groq, interleaved with tool-call events that show the agent's reasoning process in real time.

2.3 Data Flow: From Source Code to Graph

The ingestion pipeline proceeds in four stages:

1. **File Discovery:** The Indexer walks the repository directory and collects all .java and .py files, skipping build artifacts and vendor directories.
2. **AST Parsing:** Tree-sitter parses each file into a concrete syntax tree. The language-specific parser (JavaParser / PythonParser) walks the tree and emits CodeNode and CodeEdge objects for every class, method, call, import, and branch.
3. **Graph Ingestion:** The Neo4jClient bulk-upserts all nodes with MERGE statements (using the uid as the unique key) and creates directional edges. Uniqueness constraints and a fulltext index are set up on first run.
4. **Domain Clustering:** NetworkX loads the structural graph from Neo4j. python-louvain partitions the graph into communities. For each community, the Groq LLM generates a short English domain name and a one-paragraph summary. Results are written back to Neo4j as :Domain nodes.

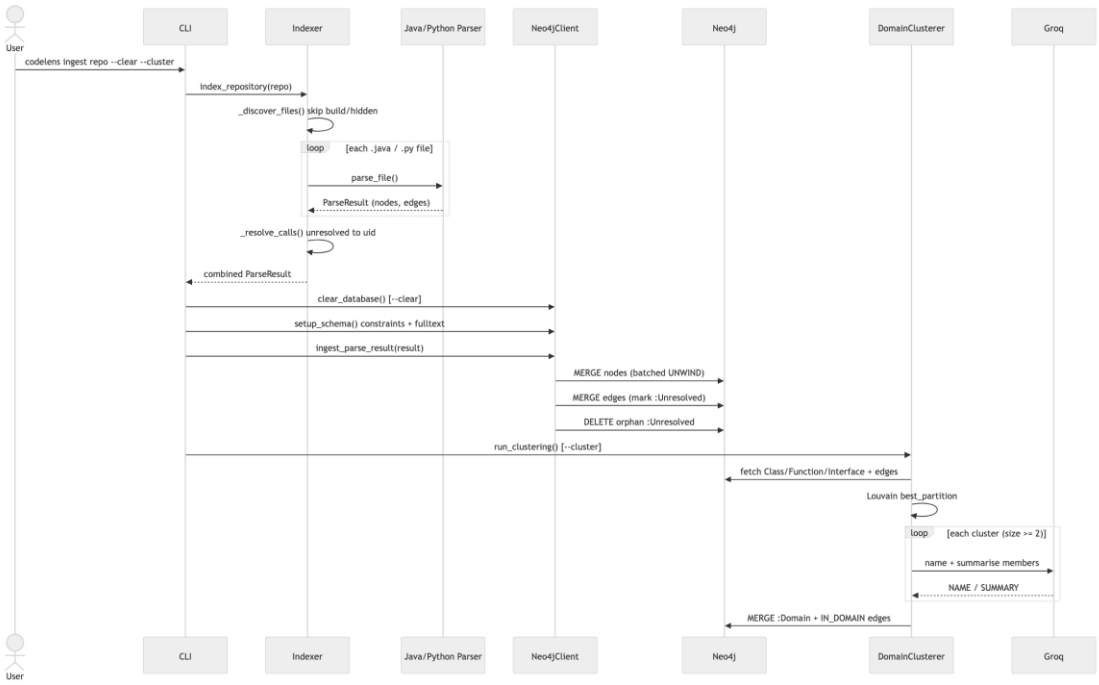

Figure 2: Ingestion pipeline sequence diagram — from codelens ingest to Domain nodes in Neo4j

3. Technology Stack

Component	Technology	Reason for Choice
AST Parsing	Tree-sitter	Fast, error-tolerant, no LSP dependency; multi-language via plugins
Graph Database	Neo4j (Docker)	Native multi-hop traversal; Cypher query language; fulltext index
Community Detection	python-louvain + NetworkX	Louvain algorithm produces stable, meaningful clusters at codebase scale
LLM — Domain Naming	Groq llama-3.3-70b-versatile	High quality output for summarization; called once per cluster only
LLM — Chat	Groq llama-3.1-8b-instant	Sub-second token latency; compatible with SSE streaming
MCP Framework	FastMCP	Minimal boilerplate; automatic tool schema generation
REST API	FastAPI + Uvicorn	Async-native; automatic OpenAPI docs; strong Pydantic integration
Authentication	JWT + bcrypt	Stateless tokens; industry-standard password hashing
Frontend	Next.js 15 App Router	React Server Components; built-in route grouping for auth separation
Graph Visualization	react-force-graph-2d	WebGL-accelerated force simulation; handles hundreds of nodes smoothly

4. Core Requirements Implementation

4.1 Multitier Architecture

The application is strictly separated into three tiers with no direct coupling between non-adjacent layers. The frontend never queries Neo4j directly; the MCP server never serves HTML; the database has no knowledge of HTTP. Communication between the presentation and application tiers uses REST (HTTP/JSON) and SSE. The MCP server uses the JSON-RPC based MCP protocol over stdio or HTTP/SSE.

4.2 User Authentication and Authorization

Authentication is implemented with JWT tokens stored as HttpOnly cookies. On login, the FastAPI backend verifies the password with bcrypt, issues a signed JWT containing the user ID, email, and role, and sets it as a cookie. Every subsequent request is validated by the get_current_user dependency via python-jose.

Two roles are enforced:

- user — access to /ask (chat) and /graph; read-only API endpoints
- admin — full access to the admin panel, user management, repository management, and domain re-clustering

The Next.js middleware reads the JWT cookie on every navigation and redirects unauthenticated or unauthorized requests to /login before the page renders.

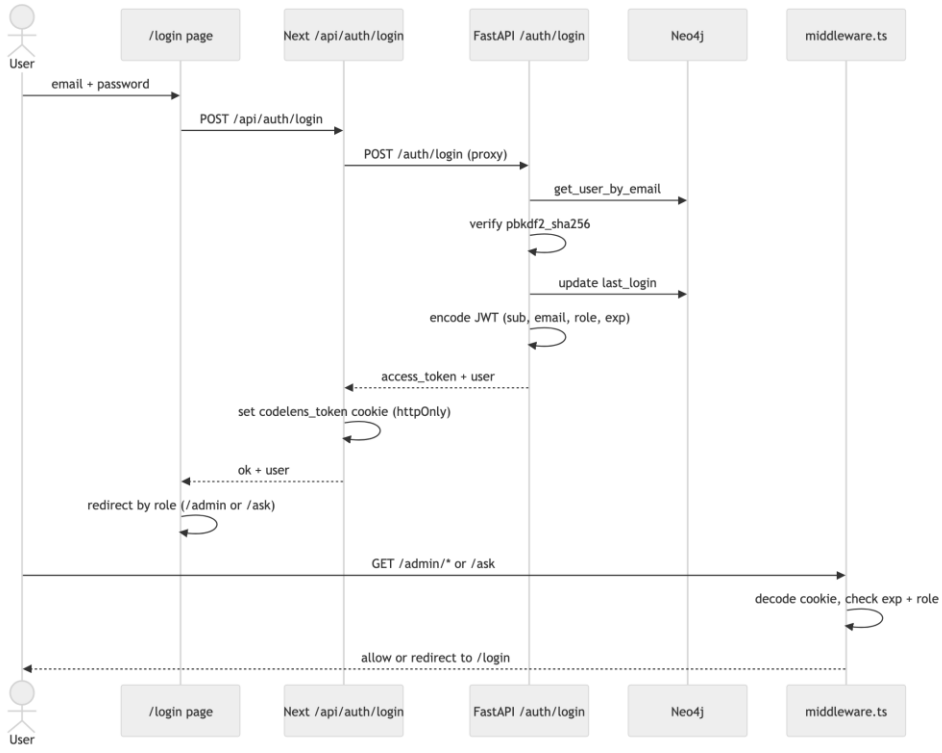

Figure 3: Login and authorization flow — from credential submission to role-based redirect

4.3 Business Logic

The core business logic is the code intelligence pipeline. Domain-specific features include:

- Multi-keyword extraction from natural language chat messages (stop-word filtered)
- Automatic graph context assembly: search → top symbol → multi-hop context → domain context
- Community detection with quality-based domain naming (skipping singleton clusters)
- Incremental re-indexing: only re-parses files whose SHA-256 hash has changed
- Background indexing jobs with status tracking (queued → indexing → succeeded/failed)

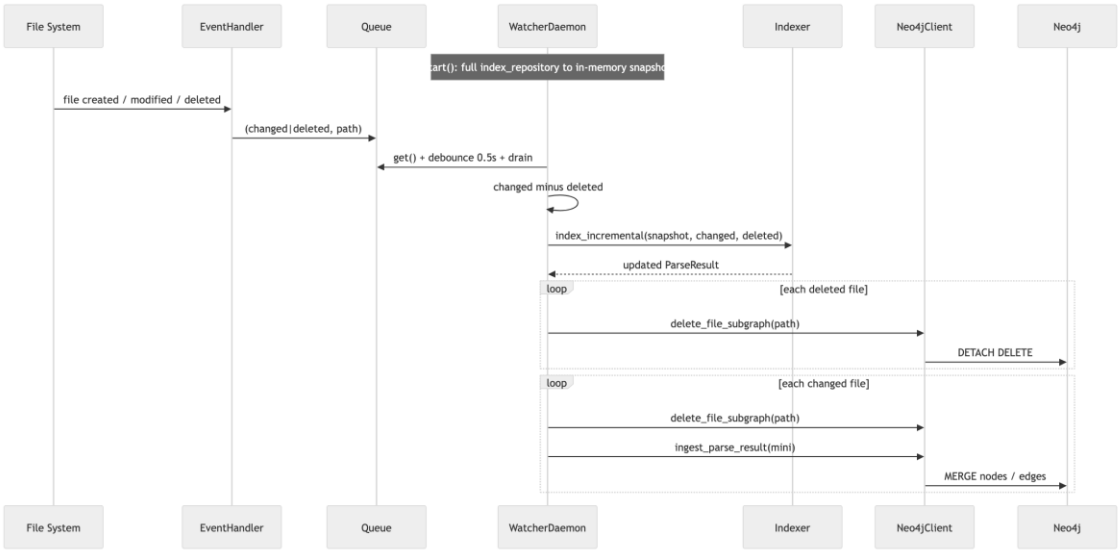

Figure 4: Incremental re-indexing flow — WatcherDaemon detects file changes and updates only affected nodes

4.4 Data Management

All data is persisted in Neo4j. The code graph uses structural labels (File, Class, Function, Interface, Constructor, ConditionalBranch, ReturnStatement, Domain). Admin data uses separate labels (User, Repository, IndexingJob) in the same Neo4j instance. Uniqueness constraints enforce data integrity on every label via the uid property. The fulltext index node_search enables Lucene-powered keyword search across names, signatures, docstrings, and file paths.

4.5 User Interface

The Next.js frontend provides three distinct interfaces:

- **/ask — Streaming Chat:**

Messages stream token-by-token via SSE. The UI renders tool call cards (showing which MCP tool was called, with what arguments, and how long it took) alongside the LLM's response. A domain sidebar lets users scope the conversation to a specific business domain.

- **/graph — Force-Directed Graph:**

Renders the Neo4j graph using react-force-graph-2d with WebGL acceleration. Nodes are colored by type; clicking a node shows its details. A repository filter dropdown limits the view to a specific ingested repo.

- **/admin/* — Admin Panel:**

Dashboard with aggregate stats, repository management (add by URL or local path, delete, re-index), domain management (list, re-cluster), and user management (create, assign roles).

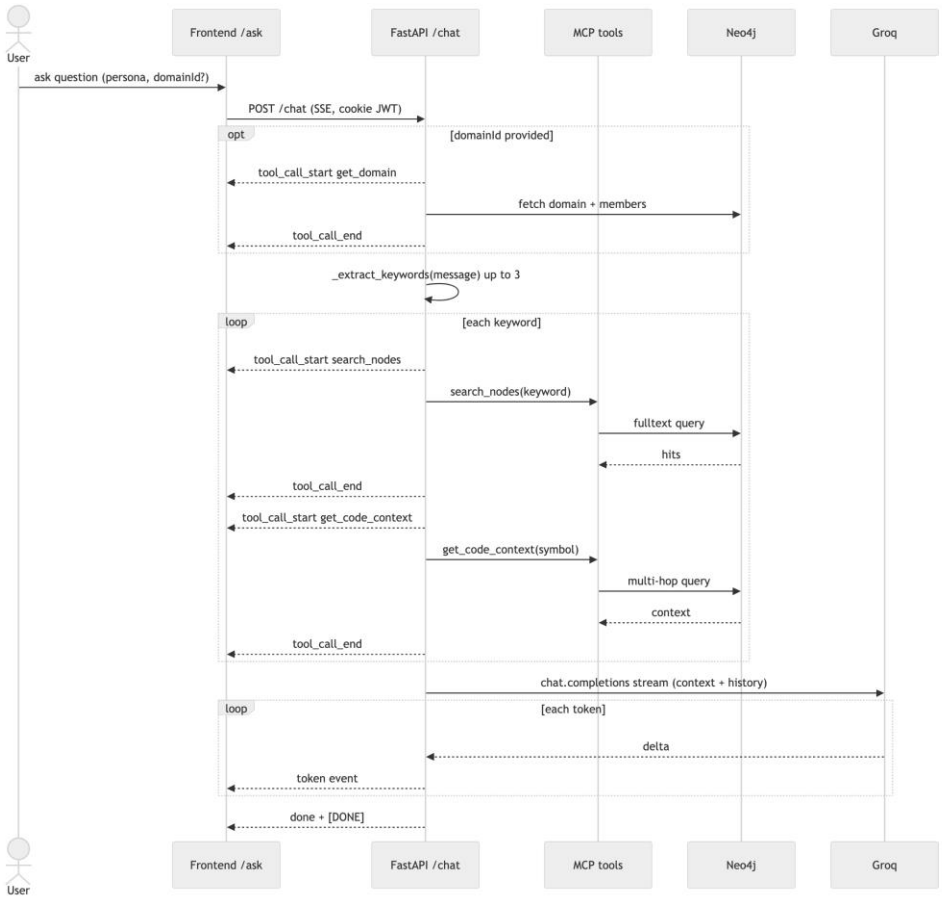

Figure 5: Chat request flow — keyword extraction, graph context assembly, and Groq SSE streaming

4.6 Third-Party API Integration

CodeLens integrates with the Groq API for two distinct purposes:

- **Domain clustering (quality model):**

After Louvain partitions the graph, one API call per cluster is made to llama-3.3-70b-versatile to generate a human-readable domain name and summary. The prompt includes the method/class signatures of all cluster members.

- **Chat responses (fast model):**

Every chat message triggers a streaming API call to llama-3.1-8b-instant. The request includes a persona-specific system prompt (developer / product / legal), the last 10 turns of conversation history, and the assembled code graph context.

Both integrations include error handling: if Groq is unavailable, domain clustering falls back to auto-generated names and chat falls back to displaying the raw graph context without LLM narration.

4.7 Security and Performance

Security measures implemented:

- Passwords hashed with bcrypt (pbkdf2_sha256 via passlib)
- JWT signed with HS256; expiry configurable via environment variable
- CORS restricted to localhost:3000 in development
- MCP stdio transport: no network exposure by default
- Neo4j credentials stored only in .env (gitignored); .env.example contains no real values

Performance optimizations:

- Fulltext index on Neo4j for sub-millisecond symbol search
- Background tasks (FastAPI BackgroundTasks) for indexing jobs — never blocks the HTTP response
- TanStack Query cache on the frontend — graph data cached for 60 seconds
- Groq streaming API — first token delivered in under 500ms

4.8 Testing

The test suite is organized into unit and integration layers using pytest with asyncio_mode = 'auto' (no manual @pytest.mark.asyncio decorators needed).

- Unit tests (tests/unit/) — validate parser output without Neo4j: node counts, edge types, signature extraction, docstring parsing
- Integration tests (tests/integration/) — validate full ingestion pipeline against a real Neo4j instance and the Spring PetClinic fixture
- CLI smoke tests — invoke codelens index via subprocess and verify output

4.9 Documentation and Presentation

Documentation delivered:

- README.md — setup instructions, architecture diagram, CLI reference, API reference, environment variables
- PROJECT_REPORT.md — full technical reference (this document's source)
- OpenAPI docs — auto-generated by FastAPI at /docs (Swagger UI) and /redoc
- Inline docstrings on all MCP tools (auto-exposed to MCP clients as tool descriptions)

5. MCP Server — Detailed Reference

The MCP server is the primary interface for AI agents. It is implemented with FastMCP and exposes 7 tools. In stdio mode, the AI agent runs the server as a subprocess; communication happens over stdin/stdout using JSON-RPC. In SSE mode, the server binds to an HTTP port and accepts connections at /sse.

Tool	Parameter	Neo4j Query Pattern	Use Case
search_nodes	keyword: str	FULLTEXT INDEX QUERY on node_search	Find entry points by name or keyword
get_code_context	symbol_name: str	OPTIONAL MATCH callers, callees, branches	Understand a symbol's full structural context
get_callers	symbol_name: str	MATCH (caller)-[:calls]->(target)	Impact analysis before changing a symbol
get_callees	symbol_name: str	MATCH (source)-[:calls]->(callee)	Understand a symbol's dependencies
get_domains	—	MATCH (d:Domain) with member count	High-level architecture overview
get_domain	symbol_name: str	MATCH (n)-[:IN_DOMAIN]->(d:Domain)	Find the business context of a symbol
get_domain_content	domain_name: str	MATCH (d:Domain {name}) + members	Explore all code in a domain

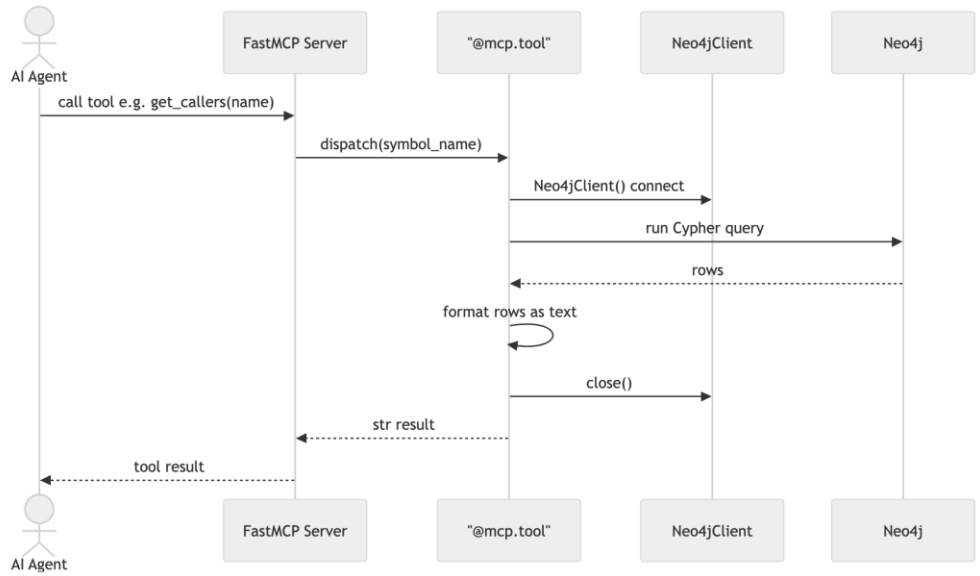

Figure 6: MCP tool query flow — AI agent call through FastMCP to Neo4j and back

6. Data Model

6.1 Code Graph Nodes

Label	Key Properties	Description
File	uid, filepath, file_hash	Source file node
Package	uid, name, filepath	Java package declaration
Class	uid, name, qualified_name, signature, docstring	Class definition
Interface	uid, name, qualified_name, signature	Java interface
Function	uid, name, qualified_name, signature, docstring, start_line, end_line	Method or function
Constructor	uid, name, signature, start_line, end_line	Constructor method
ConditionalBranch	uid, name, filepath, start_line	if/else branching point
ReturnStatement	uid, filepath, start_line	return statement
Domain	uid, name, summary, repo_id	Auto-discovered business domain

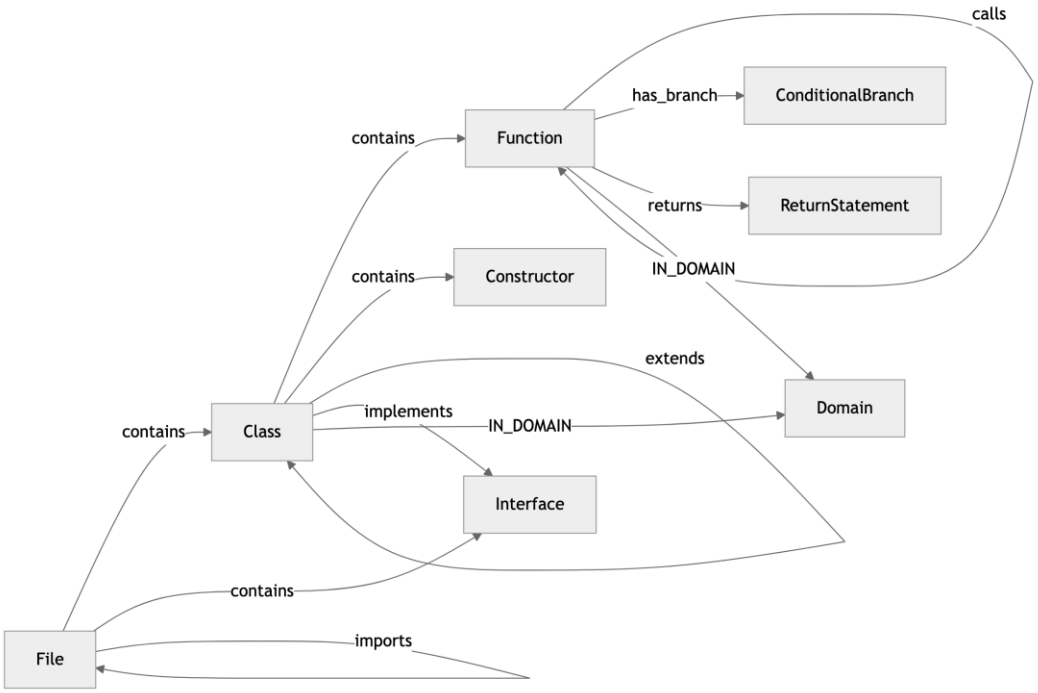

Figure 7: Neo4j data model — node types and relationships in the code graph

6.2 Relationships (Edges)

Relationship	From → To	Description
contains	File → Class/Function, Class → Function	Structural containment
calls	Function → Function	Method call

imports	File → File	Import statement
implements	Class → Interface	Interface implementation
extends	Class → Class	Inheritance
has_branch	Function → ConditionalBranch	Internal branching
returns	Function → ReturnStatement	Return point
IN_DOMAIN	Class/Function → Domain	Domain membership

6.3 Admin Data (same Neo4j instance, separate labels)

Label	Key Properties	Description
User	id, email, password_hash, role, created_at, last_login	Application user
Repository	id, name, url, paths, status, node_count, last_indexed	Registered repository
IndexingJob	id, repo_id, repo_name, status, started_at, finished_at, duration_ms, node_count, error	Indexing job record

7. Advanced Features

7.1 Full-Text Search (Advanced Data Handling)

Neo4j's native Lucene fulltext index (node_search) is created at schema setup time over Class, Function, Interface, and Constructor nodes, indexing four properties: name, docstring, signature, and filepath. This enables wildcard queries, relevance scoring, and sub-millisecond response times even on large codebases. The search_nodes MCP tool uses this index directly via CALL db.index.fulltext.queryNodes.

7.2 Real-Time Streaming (Real-Time Elements)

The /chat endpoint returns a StreamingResponse with media_type='text/event-stream'. Each SSE event is a JSON object with a type field: token (LLM output chunk), tool_call_start (tool invocation with arguments), tool_call_end (result with duration), or done. The Next.js frontend uses a custom EventSource wrapper (lib/api/sse.ts) to parse these events and render them progressively — tool call cards appear immediately while the LLM streams its answer alongside.

7.3 Analytics and Visualization

The /graph endpoint returns all nodes and edges from Neo4j as a JSON graph. The frontend renders this with react-force-graph-2d using WebGL acceleration. Nodes are color-coded by type; domain cluster membership is visible through edge groupings. The admin dashboard shows aggregate statistics: total nodes by type, total edges, indexed repository count, active domain count.

8. Setup and Deployment

8.1 Prerequisites

- Python 3.11+
- Node.js 20+ and pnpm
- Docker (for Neo4j)
- Groq API key (free tier at console.groq.com)

8.2 Local Development

```
# 1. Clone and install Python backend
git clone https://github.com/emirrozerr/code-lens.git
cd code-lens
python -m venv .venv
.venv\Scripts\activate        # Windows
pip install -e ".[dev]"

# 2. Configure environment
cp .env.example .env
# Set GROQ_API_KEY and JWT_SECRET_KEY in .env

# 3. Start Neo4j
docker compose up -d neo4j

# 4. Start the REST API
codelens api

# 5. Ingest a repository
codelens ingest tests/fixtures/spring-petclinic --clear --cluster

# 6. Start the frontend
cd frontend && pnpm install && pnpm dev
# Open http://localhost:3000
```

8.3 MCP Integration

```
# Claude Code (run once)
claude mcp add codelens -- codelens mcp

# Claude Desktop — add to claude_desktop_config.json:
{
  "mcpServers": {
    "codelens": {
      "command": "C:\\path\\to\\.venv\\Scripts\\codelens.exe",
      "args": ["mcp"]
    }
  }
}
```

9. Conclusion

CodeLens delivers a complete three-tier business application that addresses a genuine problem in modern software development: AI agents lack structural understanding of codebases. By combining AST parsing, graph database technology, community detection, and the MCP protocol, CodeLens creates an intelligence layer that makes existing AI tools dramatically more capable without replacing them.

All eight core requirements have been implemented: three-tier architecture, JWT authentication with role-based access, domain-specific business logic, Neo4j data management with fulltext search, a responsive Next.js frontend with real-time SSE streaming, Groq API integration with error handling, security via bcrypt and JWT, and a pytest test suite covering unit and integration layers.

Three advanced requirements were also addressed: full-text search with Neo4j Lucene indexes, real-time collaboration features via SSE streaming, and analytics with the force-directed graph visualization and admin dashboard.

The project is production-ready in its core pipeline. Future work could include support for additional languages (Go, TypeScript), a GitHub webhook for automatic re-indexing on push, and a hosted cloud deployment with shared team workspaces.